

Nama : Imron
NPM : 232310021
Kelas : TI24PA
Matakuliah : LAB PW TI24PA2
Praktikum 8

1. Menambahkan user management

```
return (  
  <div className="container-fluid">  
    <div className="row mb-4">  
      <div className="col-12">  
        <div className="d-flex justify-content-between align-items-center flex-wrap gap-3">  
          <div>  
            <h2 className="mb-1" style={{ color: "#437059" }}>  
              <i className="bi bi-people me-2"></i>  
              Users Management  
            </h2>  
            <p className="text-muted mb-0">Manage your user accounts</p>  
          </div>  
          <Button  
            className="d-flex align-items-center gap-2"  
            onClick={handleOpenCreate}  
            style={{ backgroundColor: "#437059",  
              borderColor: "#437059" }}  
            >  
              <i className="bi bi-plus-circle"></i>  
              Add New User  
            </Button>  
          </div>  
        </div>  
      </div>  
      <div className="row mb-4">  
        <div className="col-md-4">  
          <div className="card shadow-sm border-0">  
            <div className="card-body">  
              <h6 className="text-muted">Total  
Users</h6>  
              <h2 className="mb-0">{users?.data?.length || 0}</h2>  
            </div>  
          </div>  
        </div>  
      </div>  
    </div>  
  </div>  
)
```

```
</div>  
<div className="col-md-4">  
  <div className="card shadow-sm border-0">  
    <div className="card-body">  
      <h6 className="text-muted">Active  
Users</h6>  
      <h2 className="mb-0">  
        {users?.data?.filter((user) =>  
          user.is_active).length || 0}  
      </h2>  
    </div>  
  </div>  
</div>  
<div className="col-md-4">  
  <div className="card shadow-sm border-0">  
    <div className="card-body">  
      <h6 className="text-muted">Inactive  
Users</h6>  
      <h2 className="mb-0">  
        {users?.data?.filter((user) =>  
          !user.is_active).length || 0}  
      </h2>  
    </div>  
  </div>  
</div>  
</div>  
  
{users.loading ? (  
  <Skeleton />  
) : users.message ? (  
  <Alert message={users.message}  
variant="danger" />  
) : (  
  <Cards>  
    <Cards.Header>  
      <span className="card-label fw-bold fs-3">User Lists</span>  
    <div className="w-50">
```

```

<SearchInput
  keyword={search}
  onAction={(e) =>
setSearch(e.target.value)}
  />
</div>
</Cards.Header>
<Cards.Body className="px-0 pb-0">
  <div className="table-responsive">
    <table className="table table-hover">
      <HeaderDatatables
        headers={table_headers}
        onSorting={(field, order) => setSorting({
field, order })}
      />
      <tbody>
        {paginatedUsers.length > 0 ? (
          paginatedUsers.map((user, index) => (
            <tr key={user.id}>
              <td className="text-center">
                {(currentPage - 1) *
ITEMS_PER_PAGE + index + 1}
              </td>
              <td>
                <strong>{user.username}</strong>
              </td>
              <td>{user.email}</td>
              <td className="text-center">
                <span
                  className={`badge
${user.is_active ? "bg-success" : "bg-secondary"}}`
                >
                  {user.is_active ? "Active" :
"Inactive"}
                </span>
              </td>
              <td className="text-center">
                <Button
                  variant="warning"
                  outline
                  className="btn-sm me-2"
                  onClick={() =>
handleOpenEdit(user)}
                  title="Edit"
                >
                  <i className="bi bi-pencil"></i>
                </Button>

```

```

<Button
  variant="danger"
  outline
  className="btn-sm me-2"
  onClick={() =>
handleDelete(user.id)}
  title="Delete"
  >
    <i className="bi bi-trash"></i>
  </Button>
  <Button
    variant={user.is_active ? "warning"
: "success"}
    outline
    className="btn-sm"
    onClick={() =>
      handleToggleStatus(user.id,
user.is_active)
    }
    title={user.is_active ? "Deactivate"
: "Activate"}
    >
      <i
        className={`bi ${user.is_active ?
"bi-person-dash" : "bi-person-check"}}`
      ></i>
    </Button>
  </td>
</tr>
))
) : (
  <tr>
    <td colspan="5" className="text-
center py-5">
      <i className="bi bi-inbox fs-1 text-
muted d-block mb-3"></i>
      <p className="text-muted mb-
0">No users found</p>
    </td>
  </tr>
)}
</tbody>
</table>
{filteredUsers.length > 0 && (
  <div className="d-flex align-items-
center justify-content-center">
    <PaginationComponent

```

```

        total={filteredUsers.length}
        itemsPerPage={ITEMS_PER_PAGE}
        currentPage={currentPage}
        onPageChange={(page) =>
setCurrentPage(page)}
      />
    </div>
  )}

```

```

    </div>
  </Cards.Body>
</Cards>
)}
</div>
);
}

```

2. Validasi ketika user telah login tidak bisa akses halaman sign-in kembali

- Di backend menambahkan fungsi alreadyLoggedIn pada middleware/auth.js untuk menolak request API login/register jika pengirim sudah memiliki token valid. Frontend sudah tidak akan mengirim request jika user sudah login

```

exports.alreadyLoggedIn = (req, res, next) => {
  try {
    const authHeader = req.headers["authorization"];
    const token = authHeader && authHeader.split(" ")[1];

    if (token) {
      jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
        if (!err) {
          return res.status(403).json({
            success: false,
            message: "You are already logged in. Cannot access this endpoint.",
          });
        }
        next();
      });
    } else {
      next();
    }
  } catch (error) {
    next(error);
  }
};

```

- Di routes/userRoutes.js masukkan fungsi middleware alreadyLoggedIn pada endpoint login dan register. alreadyLoggedIn mengecek header Authorization. Jika ada token valid, server mengembalikan 403 Forbidden dengan pesan bahwa user sudah login. Dengan ini, meskipun frontend mencoba mengirim request login/register lagi (secara manual atau lewat script), akan ditolak.

```

const express = require("express");
const router = express.Router();
const userController = require("../controllers/userController");
const { verifyToken, alreadyLoggedIn } = require("../middleware/auth");

router.post("/login", alreadyLoggedIn, userController.loginUser);
router.post("/register", alreadyLoggedIn, userController.createUser);

```

```

router.use(verifyToken);

router.get("/", userController.getAllUsers);
router.get("/:id", userController.getUserById);
router.post("/", userController.createUser);
router.put("/:id", userController.updateUser);
router.patch("/:id", userController.patchUser);
router.delete("/:id", userController.deleteUser);

module.exports = router;

```

- Dalam components/auth/withAuthRedirect.jsx pada frontend terdapat pengecekan menggunakan useEffect yang memantau status user. Begitu terdeteksi pengguna sudah login (karena token masih tersimpan dan belum kedaluwarsa), useEffect akan langsung me-redirect browser ke halaman /cms/books. Hal ini membuat pengguna yang sudah login tidak bisa mengakses halaman sign-in kembali.

```

useEffect(() => {
  if (!loading && user) {
    router.push("/cms/books");
  }
}, [user, loading, router]);

```

3. Hasil

CMS Ready+ <

Dashboard
Books
Users
Testing

N

Users Management

Manage your user accounts

Add New User

Total Users
2

Active Users
1

Inactive Users
1

User Lists

NO	USERNAME	EMAIL	STATUS	ACTIONS
1	Tokisaki Kurumi	kurumi@mail	Active	
2	Violet Evergarden	1@e.com	Inactive	

« < 1 > »